

TERNARY SoC EMULATOR

Dokumentacja Techniczna i Architektoniczna

Język implementacji	Java 17+ / JavaFX
Architektura procesora	Balanced Ternary, Load/Store
Szerokość słowa	12 trytów (trit-12)
Rozmiar RAM	531 441 komórek (od -265720 do +265720)
Data dokumentacji	02.06.2026

Projekt realizuje pełną emulację 12-trytowego procesora zbalansowanego ternarnego wraz z asemblerem i interaktywnym debuggerem. System operuje całkowicie na symetrycznej arytmetyce trójkowej zamiast tradycyjnej binarnej, wspierając kompleksowy zestaw instrukcji arytmetycznych, pamięciowych oraz sterowania przepływem.

SPIS TREŚCI

1. Przegląd Projektu
 - 1.1 Cel i zakres
 - 1.2 Architektura systemu – diagram warstw
 - 1.3 Struktura plików i pakietów
2. Podstawy Arytmetyki Zbalansowanej Trójkowej
 - 2.1 Czym jest trit?
 - 2.2 Reprezentacja liczb – zakres i kodowanie
 - 2.3 Właściwości matematyczne
3. Moduł SoC.java – Rdzeń Procesora
 - 3.1 Rejestry sprzętowe
 - 3.2 Organizacja pamięci RAM
 - 3.3 Format słowa instrukcji
 - 3.4 Cykl rozkazowy (fetch-decode-execute)
 - 3.5 Flagi procesora (Status, Overflow, HLT)
 - 3.6 Kompletna lista instrukcji
4. Moduł Interpreter.java – Asembler
 - 4.1 Faza parsowania i optymalizacji
 - 4.2 Walidacja składni
 - 4.3 Kodowanie instrukcji do RAM
 - 4.4 Dyrektywy asemblera
5. Moduł GUIMain.java – Interfejs Graficzny
 - 5.1 Układ ekranu i panele
 - 5.2 Pasek narzędzi – przyciski akcji
 - 5.3 Edytor kodu źródłowego
 - 5.4 Panel stanu procesora
 - 5.5 Podgląd stosu (Stack Explorer)
 - 5.6 Mapa pamięci RAM
 - 5.7 Dziennik zdarzeń (Event Log)
 - 5.8 Debugger – breakpointy i tryb krokowy
 - 5.9 Wizualizacja graficzna trytów
 - 5.10 Obsługa plików (Open/Save/Save As)
 - 5.11 Okno pomocy (System Manual)
6. Moduł Main.java – Wejście Aplikacji
7. Moduł SystemInfo.java – Informacje Systemowe
8. Instrukcje Asemblera – Pełna Specyfikacja
 - 8.1 Instrukcje arytmetyczno-logiczne
 - 8.2 Instrukcje pamięciowe i rejestrowe
 - 8.3 Operacje stosowe
 - 8.4 Instrukcje sterowania przepływem i skoki
 - 8.5 Instrukcje systemowe
9. Przykładowe Programy i Przypadki Użycia
 - 9.1 Demonstracja skoków warunkowych
 - 9.2 Adresowanie absolutne i dyrektywy
 - 9.3 Ciąg Fibonacciego
10. Zabezpieczenia i Obsługa Błędów
11. Wymagania Techniczne

1. PRZEGLĄD PROJEKTU

1.1 Cel i zakres

Projekt Ternary SoC Emulator to w pełni funkcjonalna emulacja procesora opartego na zbalansowanej arytmetyce trójkowej (ang. Balanced Ternary). W przeciwieństwie do konwencjonalnych komputerów operujących na bitach (wartości 0 i 1), ten procesor operuje na trytach (ang. trits) przyjmujących wartości {-1, 0, +1}.

Zakres projektu obejmuje:

- Rdzeń procesora (SoC) z pełnym zestawem 27 unikalnych instrukcji,
- Zbalansowaną pamięć RAM o zakresie adresowym od -265 720 do +265 720,
- Asembler inline tłumaczący tekst mnemoniki na słowa binarne w RAM,
- Graficzny interfejs użytkownika (JavaFX) z debuggerem krokowym,
- Wizualizację rejestrów, stosu i pamięci w postaci kolorowych bloków trytów,
- System breakpointów, dziennika zdarzeń i obsługi plików .txt.

1.2 Architektura systemu – diagram warstw

Warstwa	Komponent	Rola w systemie
Prezentacji	GUIMain.java	Interfejs JavaFX: pole edytora, siatki pamięci, debugger krokowy.
Logiki i Translacji	Interpreter.java	Parsowanie źródeł, kontrola składni, wstrzykiwanie do RAM.
Sprzętowa	SoC.java	Emulacja sprzętu: Rdzeń, matryca RAM oraz cykl rozkazowy.
Systemowa	SystemInfo.java	Kontrola środowiska: Java Virtual Machine oraz OpenJFX.
Wejścia	Main.java	Kontener dla dawnego środowiska CLI (obecnie w pełni przejęte przez GUI).

1.3 Struktura plików i pakietów

Wszystkie klasy operacyjne należą do głównego pakietu javaproject.

Plik	Klasa	Rola w systemie
SoC.java	SoC	Definiuje komórki pamięci, rejestry oraz logikę jednostki ALU.
Interpreter.java	Interpreter	Komulator tłumaczący tekst źródłowy na natywne słowa maszyny.
GUIMain.java	GUIMain	Inicjalizuje układ okien i łączy pola interfejsu z silnikiem obliczeniowym.
Main.java	Main	Zapasowy punkt wejściowy przechowujący pętle testowe CLI.

Plik	Klasa	Rola w systemie
SystemInfo.java	SystemInfo	Pobiera i przekazuje informacje o systemie wykonawczym JVM.

2. PODSTAWY ARYTMETYKI ZBALANSOWANEJ TRÓJKOWEJ

2.1 Czym jest trit?

Trit (ang. ternary digit) to podstawowa jednostka informacji w architekturze trójkowej. W systemie zbalansowanym odwzorowuje trzy symetryczne stany fizyczne:

Symbol	Wartość numeryczna	Reprezentacja graficzna w interfejsie
+	+1 (Stan dodatni)	Zielony blok (#238636)
0	0 (Stan neutralny)	Szary blok (#30363d)
-	-1 (Stan ujemny)	Czerwony blok (#da3633)

2.2 Reprezentacja liczb – zakres i kodowanie

Standardowy rejestr sprzętowy posiada szerokość 12 trytów. Przekłada się to na obsługę liczb całkowitych w zbalansowanym zakresie od -265 720 do +265 720. Maksymalne granice wynikają bezpośrednio z matematyki potęgowania o podstawie 3:

```
max_value = (3^12 - 1) / 2 = (531 441 - 1) / 2 = 265 720
```

Funkcja sprzętowa `truncat12(value)` na bieżąco normalizuje wynik każdej operacji do tego okna, realizując bezpieczne owijanie (wrapping) symetryczne:

```
mod_result = (value + 265720) % 531441
```

```
final_bounded_value = mod_result - 265720 (korekcja przesunięcia do zera)
```

2.3 Właściwości matematyczne

- Pełna symetria wokół zera – całkowicie eliminuje problem sprzętowego dublowania zera oraz skomplikowanego przedłużania znaków (jak w systemie binarnego U2).
- Stan neutralny (0) pozwala na natywne reprezentowanie wartości pustych, nieprzypisanych lub logicznego oczekiwania.
- Przesunięcie trytowe (SHF) idealnie skaluje wartości matematyczne, mnożąc i dzieląc przez bezpośrednie potęgi trójki.
- Zjawisko przepełnienia (overflow) jest symetryczne, oddzielając nadmiar wymuszony wartością dodatnią (+1) od wartości ujemnej (-1).

3. MODUŁ SoC.java – RDZEŃ PROCESORA

Moduł SoC przechowuje główny model wirtualnego procesora. Rezerwuje pamięć operacyjną, definiuje strukturę rejestrów, steruje logiką jednostki ALU oraz zarządza cyklem instrukcji (fetch-decode-execute).

3.1 Rejestry sprzętowe

Nazwa	Rozmiar	Przeznaczenie architektoniczne	Wartość po resecie
PC	12 trytów	Program Counter – Wskazuje adres kolejnej instrukcji.	START_ADDRESS = -265 720
SP	12 trytów	Stack Pointer – Kontroluje szczyt wskaźnika stosu pamięciowego.	MAX_VALUE = +265 720
R-	12 trytów	Rejestr ogólnego przeznaczenia (Indeks symetryczny: -1).	0
R0	12 trytów	Główny akumulator przestrzeni operacyjnej (Indeks symetryczny: 0).	0
R+	12 trytów	Rejestr ogólnego przeznaczenia (Indeks symetryczny: +1).	0
STS	1 tryt	Status Flag – Loguje wartość wyjściową operacji (-1=Neg, 0=Zero, +1=Pos).	0
OVR	1 tryt	Overflow Flag – Informuje o przekroczeniu zakresu (-1 / 0 / +1).	0
HLT	boolean	System State Flag – Zatrzymuje pętlę wykonawczą po przejściu w true.	false

3.2 Organizacja pamięci RAM

Fizyczna pamięć to tablica o całkowitym rozmiarze 531 441 komórek. Ich adresowanie obejmuje zbalansowany logiczny zakres od -265 720 do +265 720. Co krytyczne, każda pojedyncza komórka mieści w sobie jedynie 6 trytów (stanowiąc równą połowę 12-trytowego słowa maszyny).

Matematyczne rzutowanie adresów logicznych na fizyczne indeksy tablicy:

```
array_index = (logical_address + MAX_VALUE) % ramSize
```

Z racji tych wymiarów, operacje 12-trytowe (LDR, STR, PSH, POP, CALL, RET) automatycznie pokrywają zawsze dwie sąsiadujące ze sobą komórki RAM, aby zapisać lub odczytać pełną wartość:

```
writeRam12(address) → RAM[address] = upper_half, RAM[address+1] = lower_half
```

3.3 Format słowa instrukcji

Zgodnie z powyższym, każda instrukcja maszynowa musi zająć idealnie dwie ciągłe komórki RAM (łącznie 12 trytów):

Blok pamięci	Nazwa przydziałowa	Wewnętrzny podział trytowy / bitowy
RAM[PC]	Command Word	[Nieużywane/0][Opcode (3 Tryty)][Target Rd (1 Tryt)][Source Rs (1 Tryt)]
RAM[PC+1]	IMM Data Word	Przechowuje 6-trytową stałą (Pole całkowitoliczbowe z zakresu [-364, +364])

Główne słowo polecenia jest upakowywane do RAM za pomocą specjalnego wzoru: $\text{cmdWord} = (\text{opcodeId} * 9) + (\text{rd} * 3) + \text{rs}$, gdzie opcodeId zamyka się w przedziale $[-13, +13]$, a kody rejestrów docelowych (rd) i źródłowych (rs) zajmują wartości $\{-1, 0, +1\}$.

3.4 Cykl rozkazowy (fetch-decode-execute)

Faza	Docelowe zadanie cyklu	Odzwierciedlenie w kodzie Java
1. Pobranie komendy	Odczyt pierwszego słowa z aktualnego adresu PC.	<code>cmdWord = readRam(this._programcounter);</code>
2. Pobranie danych	Odczyt słowa z danymi natychmiastowymi pod PC+1.	<code>imm = readRam(this._programcounter + 1);</code>
3. Przesunięcie PC	Inkrementacja Program Counter do następnego bloku RAM.	<code>_programcounter += 2;</code>
4. Normalizacja	Usunięcie ujemnych reszt w celu poprawnego działania modulo.	<code>shifted = cmdWord + 364;</code>
4a. Wyodrębnienie Rs	Rozpakowanie indeksu źródłowego rs bazą 3.	<code>rs = (shifted % 3) - 1;</code>
4b. Wyodrębnienie Rd	Rozpakowanie indeksu docelowego rd bazą 3.	<code>rd = ((shifted / 3) % 3) - 1;</code>
4c. Wyodrębnienie Op	Określenie identyfikatora ID bazowej instrukcji.	<code>opcodeId = ((shifted / 9) % 27) - 13;</code>
5. Wykonanie	Przekazanie sterowania do bloku obsługi konkretnego polecenia.	<code>executeDecoded(opcode, rd, rs, imm);</code>

3.5 Flagi procesora

Flaga	Szerokość	Stany natywne	System operacyjny wyzwalania
STS (Status)	1 tryt	+1 = Wynik > 0 0 = Wynik == 0 -1 = Wynik < 0	ADD, SUB, ADC, SBC, CMP, MIN, MAX, INV, SHF
OVR (Overflow)	1 tryt	+1 = Limit dodatni 0 = W normie -1 = Limit ujemny	Zmieniana przez wszystkie układy ALU funkcją <code>updateFlags()</code>
HLT (Halt)	boolean	true = Całkowite zawieszenie pętli cyklu maszynowego	Wywoływana z polecenia absolutnego HLT

3.6 Kompletna lista instrukcji

Wewnętrzne mapowanie tablicy dekodera dla sprzętowej siatki wykonawczej:

Instrukcja	Kod ID	Instrukcja	Kod ID
SUB	-13	SHF	-12
ADD	-11	SBC	-10
CMP	-9	ADC	-8
MIN	-7	INV	-6
MAX	-5	LDR	-4
LLI	-3	LHI	-2
STR	-1	MOV	0
PSH	1	POP	2
PSF	3	POF	4
JMN	5	OVN	6
JMP	7	WSP	8
JMZ	9	HLT	10
CALL	11	OVP	12
RET	13	—	—

4. MODUŁ Interpreter.java – ASSEMBLER

Klasa Interpreter realizuje warstwę kompilatora inline. Pobiera surowe ciągi tekstowe z okna edytora, usuwa zanieczyszczenia składniowe, weryfikuje tokeny logiczne i generuje na ich podstawie wykonywalne słowa binarne prosto do komórek docelowej matrycy RAM.

4.1 Faza parsowania i optymalizacji

Statyczna funkcja `optimize(String line)` czyści wprowadzony kod według zasady:

- Usuwa wtrącenia tekstowe użytkownika i komentarze zaczynające się od tradycyjnych spacji i ukośników `//`,
- Usuwa białe znaki tabulacji z końców instrukcji oraz ukryte formatowania systemowe ze schowka,
- Kondensuje puste i nieprzewidywalne przerwy między parametrami poleceń w równe, pojedyncze spacje separatora.

4.2 Walidacja składni

Po rozłożeniu tekstu pętla `validateSyntax(String[] parts)` mierzy układ parametrów i rygorystycznie weryfikuje przypisanie z tablicami logicznymi:

Zbiór kolekcji	Dozwolone polecenia macierzyste	Strukturalny wymóg parametru wejściowego
NO_ARG	HLT, PSF, POF, RET	Ścisły brak dodatkowych parametrów (Zezwolono jedynie na Opcode).
RD_ONLY	INV, PSH, POP, JMN, OVN, JMP, WSP, CALL, OVP, JMZ	Wymagany dokładnie 1 parametr (Wymaga przekazania rejestru docelowego Rd).
RD_RS	SUB, SHF, ADD, SBC, CMP, ADC, MIN, MAX, LDR, STR, MOV	Wymagane dokładnie 2 parametry (Rejestr docelowy Rd, po którym następuje źródłowy Rs).
RD_IMM	LLI, LHI	Wymagane dokładnie 2 parametry (Rejestr docelowy Rd i wartość liczbowa Immediate).

4.3 Kodowanie instrukcji do RAM

Mechanizm mapowania RAM `assembleAndLoad(List programLines)` iteruje po zaakceptowanym pliku i osadza bloki danych:

1. Prześwietla komendę przez optymalizator usuwający komentarze.
2. Wychwytuje dyrektywy nadpisania początku programu zawarte w `[n]` i aktualizuje licznik `currentRamAddress`.
3. Transformuje stringi mnemoniczne i ekstrahuje ID dla pól logicznych: `opcodeId`, `rd`, `rs`, `imm`.
4. Oblicza finalną strukturę dla RAM: `commandWord = (opcodeId * 9) + (rd * 3) + rs`.
5. Wstrzykuje warstwy prosto do komórek: `cpu.injectRAM(addr, cmd) + cpu.injectRAM(addr+1, imm)`.

6. Zwiększa wskaźnik `currentRamAddress += 2` (ponieważ każde polecenie alokuje fizycznie dwa rzędy matrycy).

4.4 Dyrektywy asemblera

Dyrektywa Mnemoniczna	Przykładowy zarys składni	Zastosowanie w przestrzeni kompilacji
Komentarz Linijkowy	// Notatka techniczna	Nakazuje wyciąć wszystkie teksty występujące za znakiem z dalszego przetwarzania.
Komentarz Blokowy	# Notatka techniczna	Każe skanerowi asemblerowemu kompletnie zignorować zawartość tej linijki tekstu.
Skok Origin RAM	[target_address]	Zmusza sprzętowy wskaźnik ładujący asemblera do przeskoczenia bezpośrednio w nową celowaną komórkę RAM.
Flaga Breakpoint	* nazwa_polecenia	Nanosi flagę śledzącą dla danej komórki. Interpreter zatrzyma logikę maszyny przed odczytem danego opkodu.

5. MODUŁ GUIMain.java – INTERFEJS GRAFICZNY

Zadaniem komponentu GUIMain jest budowanie wysoce interaktywnego, przejrzystego interfejsu aplikacji. Tworzy on środowisko deweloperskie oparte na jasnym motywie kolorystycznym, które odsłania wewnętrzne mechanizmy procesora do ciągłego wizualnego śledzenia w trybie Live.

5.1 Układ ekranu i panele

Nazwa panelu operacyjnego	Pozycja w środowisku	Zadanie zintegrowane
Pasek narzędzi (Toolbar)	Obszar szczytowy	Mieszczą się w nim wyzwalacze globalne, sterowanie I/O i wskaźniki błędów.
Lewa kolumna informacyjna	Lewa krawędź	Udostępnia podgląd stosu operacyjnego (Stack) i pełne wartości wszystkich rejestrów maszynowych.
Centralne okno wskaźnikowe	Przestrzeń środkowa	Mapuje wybrane bloki komórek matrycy RAM w formacie czasu rzeczywistego (Live Tracking).
Prawa kolumna wejściowa	Prawa krawędź	Kontroluje surowy log błędów assemblerowych i udostępnia obszar do bezpośredniego pisania programów.

5.2 Pasek narzędzi – przyciski akcji

Makro element paska	Przypisany Skrót	Systemowy skutek wywołania akcji z elementu interfejsu
☐ RESTART	–	Czyszczenie silnika: Resetuje assembler, uszkadza logi wejścia, odświeża cykl na zerowych wartościach rejestrów.
? MANUAL	–	Otwiera nowe oddzielne okno instruktażowe z pomocą dla zagubionych i specyfikacją procesora w 4 zakładkach.
☐ OPEN	Ctrl + O	Zmusza FileChooser do podpięcia lokalnych skryptów z dysku (tylko *.txt) wewnątrz edytora tekstowego maszyny.
☐ SAVE	Ctrl + S	Nadpisuje kod na otwartym dotąd obiekcie, bądź automatycznie wymusza ścieżkę nadpisania okna przez procedurę Zapisz Jako.
☐ Dropdown Menu Arrow	–	Odsłania ukryty parametr menu dedykowany oddzielnemu wywołaniu Save As (skrót klawiszowy Ctrl+Shift+S).
☐ SYNTAX CHECK	–	Nakazuje na sucho zweryfikować składnię. Wykrywa wszystkie braki, ale powstrzymuje samą fazę RAM Injection wykonania.
☐ STEP	–	Skutkuje fizycznym przepuszczeniem dokładnie 1 (jednego) obrotu jednostki cyklowej. Bardzo przydatne do naprawiania błędów w pętlach.

Makro element paska	Przypisany Skrót	Systemowy skutek wywołania akcji z elementu interfejsu
☐ RUN FULL	–	Zwalnia zabezpieczenia i realizuje cykl maszynowy w pełni aż do wywołania HLT, breakpointa lub przebiccia guard limitu 1500 operacji.

5.3 Edytor kodu źródłowego

Prawy panel zawiera edytor tekstowy (TextArea). Posiada on zoptymalizowaną podpowiedź w tle, która wyświetla minimalistyczny szablon kodu, gdy edytor jest pusty:

```
LLI R0 10 LLI R+ 5 ADD R0 R+ *CMP R0 R+ HLT
```

5.4 Panel stanu procesora (Processor State)

Panel ten udostępnia dane sprzętowe dla każdego dostępnego na płycie rejestru maszynowego w strukturze połączonego podglądu tekstowo-graficznego. Pojawiają się na nim wskaźniki dziesiętne liczbowe oraz kaskady dokładnie ułożonych 12 kolorowych bloków fizycznych trytów. Dla wyróżnienia i czytelności wskaźniki sprzętowe sterowania (PC, SP) prezentowane są etykietami na niebiesko (#58a6ff), obszary danych (R-, R0, R+) otrzymują obrys szarości (#c9d1d9), z kolei same flagi logiczne operacji (STS, OVR, HLT) wypuszczane są jako wyróżnione akcenty fioletowe (#d2a8ff).

5.5 Podgląd stosu (Stack Explorer)

Obszar ten bezpośrednio loguje alokację warstw podprzestrzeni wykonawczej typu Call Stack. System śledzi dokładnie 8 kaskadowo przypiętych adresów bazowych wychodzących w górę od adresu określonego sztywnym indeksem SP. Dzięki rosnącej ujemnie zbalansowanej architekturze przestrzeni, operacja PSH pomniejsza licznik wskaźnika komórkowego (-), co naturalnie zrzuca stos fizycznie w dół przy prezentacji dla analityka.

5.6 Mapa pamięci RAM (Memory Map)

Środkowy podgląd matrycy ujawnia zintegrowane widoki okna celowniczego rozciągniętego na 15 rzędów roboczych wokół aktywnego wiersza wskazywanego przez PC. Dla zdefiniowanego kompromisu pomiędzy wygodą czytania a wielkością okna użyto logowania wektorowego w modelu:

```
[ADRES KOMÓRKI RAM] | INTEGER DZIESIĘTNY | [BLOKI 6 TRYTÓW TERNARNYCH] << PC | [B]
```

Znacznik << PC w asyście odpowiedniej ramy podświetla dokładnie aktualny rząd odczytowy. Tymczasem flaga boczna [B] natychmiast przypomina analitykowi w środowisku o nałożeniu breakpointu zatrzymującego w tym miejscu RAM.

5.7 Dziennik zdarzeń (System Event Log)

Komponent terminala błędów pozwala natychmiast logować zdarzenia pomyślne, błędy składni poleceń `[ERROR Ln]` i powiadomienia o kolosalnych naruszeniach alokacyjnych i wyjściu Program Countera przez limit `[KERNEL PANIC]`, jak również informuje u spodu widoku o poprawnym złapaniu aktywnego markera powstrzymującego `[BREAKPOINT]`.

5.8 Debugger – breakpointy i tryb krokowy

Skaner potrafi rozpoznać umiejscowienie modyfikatorów kontroli śladu wstawiając znacznik znaku *, dodając logowane adresy pod mapę poszukiwań `breakpointsAddrs`. Daje to bezpośrednią siłę wywoławczą podpinając się dwiema oddzielnymi konfigurowalnymi gałęziami:

Środowiskowy tryb Debug	Modyfikatory kontrolera flag	Zachowanie pętli operacyjnej środowiska wykonawczego
Krok po kroku (STEP)	stepMode = true	Błyskawicznie wykonuje wyłącznie jedno wskazane aktualnie polecenie i blokuje obwód (maxExecutionLimit spada do 1).
Ciągłe wykonanie (RUN)	stepMode = false	Przetwarza nieprzerwanie aż nie spotka instrukcji HLT, wskaźnika adresowego Breakpoint lub uderzy graniczny licznik 1500 obrotów bezpieczeństwa (Timeout).

Rejestr zablokowania pamięci maszynowej na runPaused pozwala w locie wznowiać operacje omijając przerwany cykl pracy polecenia bez obowiązku kasowania logów pamięci operacyjnej.

5.11 Okno pomocy (System Manual)

Modal pomocniczy zawierający cztery ustrukturyzowane podsekcje nakierowane precyzyjnie na wspomaganie użytkownika w programowaniu:

- Szybki Start – Instrukcja inicjalizacyjna prezentująca znaczenie profilów barw rzutowania i zasady operacyjne.
- Architektura – Dokładny opis sprzętu technicznego maszyny, struktury dzielonych na obydwie połówki plików RAM oraz list adresowych.
- Instrukcje – Obszerny wyciąg ze wszystkimi dostępnymi dla systemu mnemonicznymi hasłami i modyfikatorami flag operacji.
- Przykłady – Przygotowane logicznie pełne zarysy działających obwodów obliczeniowych, skoków warunkowych i testów z ciągami na strukturach podziału.

6. MODUŁ Main.java – WEJŚCIE APLIKACJI

Plik klasowy maszyny w formie Main odpowiada pierwotnie za wejście dla terminalowego ujęcia komendowego CLI. Pojemnik z testowymi skryptami tekstowymi zamknięto z tyłu pod osłoną komentarzy blokowych. W aktualnym modelu produkcyjnym zrzuca ono obroty testowe i wywołuje polecenie załadowania z klasyczną formą bezpośredniego połączenia za pośrednictwem modułów JavaFX jako `GUIMain.main()`.

7. MODUŁ SystemInfo.java – INFORMACJE SYSTEMOWE

Zapasowa statyczna paczka testowa diagnozująca SystemInfo potrafi zbierać z otoczenia systemowego natychmiast parametry do określenia właściwości bibliotek uruchomionego modułu. Udostępnia dwa kluczowe ciągi informacyjne w formie publicznych właściwości odczytu:

Statyczna Metoda Diagnostyczna	Wyciągane środowiskowe zasoby informacyjne JVM
<code>javaVersion()</code>	Weryfikuje pole wejściowe maszyny testowej wyciągając pole z <code>System.getProperty("java.version")</code> .
<code>javafxVersion()</code>	Odsłania pakiet dostarczonego obwodu interfejsu graficznego powiązanego z polem <code>System.getProperty("javafx.version")</code> .

8. INSTRUKCJE ASEMBLERA – PEŁNA SPECYFIKACJA

8.1 Instrukcje arytmetyczno-logiczne

Mnemonic	Syntax assembly	Operational hardware action	Flags affected
ADD	ADD Rd Rs	$Rd = Rd + Rs$ (Signed addition and subtraction)	STS, OVR
SUB	SUB Rd Rs	$Rd = Rd - Rs$ (Operation of symmetric subtraction of mathematical)	STS, OVR
ADC	ADC Rd Rs	$Rd = Rd + Rs + OVR$ (Flags assigned from above overflow range violations)	STS, OVR
SBC	SBC Rd Rs	$Rd = Rd - Rs + OVR$ (Carry with use of coupled negative status of OVR)	STS, OVR
CMP	CMP Rd Rs	Calculate $STS = \text{sign}(Rd - Rs)$. Targets same undergo protection and objectively not modify themselves	STS, OVR
MIN	MIN Rd Rs	$Rd = \min(Rd, Rs)$ (Scales vector to minimum indicated as lowest value)	STS, OVR
MAX	MAX Rd Rs	$Rd = \max(Rd, Rs)$ (Takes from circuit hardware logic dimension input)	STS, OVR
INV	INV Rd	$Rd = -Rd$ (Reverses precisely symmetry on all block components on their opposite)	STS, OVR
SHF	SHF Rd Rs	Scales blocks in ternary model according to operation: Multiplication ternary when $Rs > 0$ and reverse division when $Rs < 0$	STS, OVR

8.2 Instrukcje pamięciowe i rejestrowe

Mnemonic	Syntax assembly	Functional profile and operational system Load/Store
LLI	LLI Rd IMM	Prepumps given input size immediately under bottom structure 6-ternary block for Rd, isolating safely the rest of top.
LHI	LHI Rd IMM	Applies indicated components inside immediately ternary pool directly to higher ternary operations maintaining safely bottom on place.
LDR	LDR Rd Rs	Writes component machine parameters directly from read from target personalized relative pointer in cell of Rs to same Rd.
STR	STR Rd Rs	Traverses full circuit package cells with total parameter from line of target Rd to hardware addressable set from Rs.
MOV	MOV Rd Rs	Duplicates full information width of assigned component registers physically directly from Rs transferring their logical view to Rd.

Notatka dotycząca architektury: Sprzętowy wpis potężnej 12-trytowej struktury parametrów musi korzystać ze zgodnych logicznych dwuetapowych konfiguracji poprzez pary połączonych mechanizmów z funkcjonalności rozszerzonej na LHI połączonym idealnie dla bazy przez LLI. Na przykład weryfikowalne uzbrojenie docelowej stacji akumulatora na R0 rzutowanej z pulą zasobu logiki wprost pod stałą na cel 1000 sprowadzi układ asemblerowy do wpisu:

```
LHI R0 1 // Ustawia wyższe metryki dla kompilatora operacji według rzutu bazy trytowej:
1 * 729 = 729
```

```
LLI R0 271 // Aktywuje ramy dla uzupełnienia po dolnym rzucie limitowym do uzyskania
precyzyjnie celowego ułożenia + 271 = 1,000
```

8.3 Operacje stosowe

Mnemonik	Składnia asemblerowa	Reaktywny cykl nałożonej komendy do mechanizmów pamięci Stack
PSH	PSH Rd	Dokonuje pomniejszonego skoku licznika przestrzennego wskaźnika komórkowego i wypycha obwody z pola ułożenia do systemu góry rejestru Stack Pointer w logice: $SP -= 2$; $RAM[SP] = Rd$.
POP	POP Rd	Odbiera operacyjną wierzchnią z góry narzuconej logiki stosowej celowej pod docelowe przeznaczenie komórek dla sprzężonego z góry rzutowania: $Rd = RAM[SP]$; $SP += 2$.
PSF	PSF	Rzutuje połączone skompresowane ramowe wytyczne dla systemu dla celów przetrzymania danych trytów informacyjnych STS w modelu pakietowanym od rzutnika OVR na: $(STS * 3) + OVR$.
POF	POF	Odzyskuje złożoną pakietowo z bitów strukturalną formację ze zdjętych szczytów stosowych przywracając oryginalne parametry OVR zachowując bezpiecznie STS w trybie wskaźnikowym dla logiki.
WSP	WSP Rd	Ogranicza całkowicie funkcje operacyjne ze zmodyfikowanych wskaźników operacji i sprowadza system na odniesienie po wskazywanym przez parametr adresowym polu docelowym u Rd na: $SP = Rd$.

8.4 Instrukcje sterowania przepływem i skoki

Mnemonik	Składnia asemblerowa	Wymagane zatwierdzenia flagi rzutu logiki do sprzętu	Opis bazowego zachowania poleceń w modelu docelowym obwodu cyklu
JMP	JMP Rd	Wyzwalacz rzutowania na polu flag pod kontrolę statusu ($STS == 1$)	Ustawia Program Counter na wytypowany systemem adres celowy na: $PC = Rd$
JMN	JMN Rd	Wyzwalacz uwarunkowany spadkiem na wytyczonej linii statusu do ujemnych ($STS == -1$)	Ustawia Program Counter na wytypowany systemem adres celowy na: $PC = Rd$
JMZ	JMZ Rd	Wyzwalacz rzutowania w punkcie symetrycznej stabilności pola do neutrum ($STS == 0$)	Ustawia Program Counter na wytypowany systemem adres docelowy na: $PC = Rd$
OVP	OVP Rd	Rozbicie limitów granicznych wyłapane przez operacyjną komorę z góry limitów u Overflow ($OVR == 1$)	Ustawia Program Counter na wytypowany systemem adres docelowy na: $PC = Rd$

Mnemonic	Syntax assembly	Required assertions of logic flags to hardware	Description of basic instruction behavior in the target circuit model
OVN	OVN Rd	Underflow compensation from operation on circuit flag limit on logic Overflow up (OVR == -1)	Set Program Counter to specified address system PC = Rd
CALL	CALL Rd	Operates in mode completely uncompromised by limit on logic and compensation error	Pushes instruction pointer to memory stack by changing read PC to Rd
RET	RET	Jump mode and unconditionally operating under exit procedural and block jumps programs	Catches memory vectors conditioned by parameter set on PC removing virtual Stack block

8.5 System instructions

Mnemonic	Syntax assembly	Description of reactive hardware block inside the circuit model called to the instruction loop under the loop in the system execution processor.
HLT	HLT	Raises and sets the circuit safety mode holding the trigger (HLT skips to true). Operates to prevent further code blocks under the safety exit.

9. PRZYKŁADOWE PROGRAMY I PRZYPADKI UŻYCIA

9.1 Demonstracja skoków warunkowych (JMZ)

Ten przykładowy program odejmuje od siebie dwie równe wartości i wykorzystuje instrukcję JMZ do ominięcia nieskończonej pętli, kończąc bezpiecznie wykonywanie.

```
LLI R0 5 // Dołączenie limitowe po 5 wywołuje cel na R0
LLI R+ 5 // Przekazanie na uwarunkowanie wsparcia ujęte wewnątrz podziału przez 5 w
docelowe po R+
SUB R0 R+ // Baza komendowa dla operacji z arytmetycznym zadaniem polecenia w Rd z R+
co osadza się w 0 na Statusie Flagi
LLI R- -265710 // Rzutu docelowo limit wektora ucieczki awaryjnej u wyłącznika
wyjściowego bazy we wskaźnikowym na R-
JMZ R- // Właściwy uwarunkowany moment przejścia warunkującego skoku na wyjściowe zero
prosto kierując krokami pod R-
HLT // Ignorowany przez maszynę na poziomie odczytów w trakcie fazy przeskokuwej
[-265710] HLT // Etykietowane adresowe wywołanie dyrektywy pod bezpośrednie
bezpieczeństwo blokujące.
```

9.2 Adresowanie absolutne i dyrektywy

Zarządzanie środowiskiem asemblera ukazuje osadzenie odrębnych wektorów ramowych pod bezpośrednie nadpisy wskaźników przez dyrektywę pod adresem ukrytą we klamrowych wyzwalaczach:

```
[-250000] // Uderzenie limitowe nadpisujące bieżącą alokację u kroków kompilacji o
wskaźnik wymuszony bezpośrednio na zadeklarowanej komórce.
LLI R0 42 // Pobór parametrów docelowych natychmiast u wartości do zagnieżdżenia
LLI R- 250 // Uwarunkowane oddanie punktowego wyjścia bazy pod wymuszony rzut w wiersz
o indeksowym celu w pamięci
STR R0 R- // Przeprowadza i obniża rzuty komórkowe poprzez przepchanie wymuszonych
ułożeń od R0 pod przypięte bazy ram
LDR R+ R- // Zawraca uwarunkowanie logiki poprzez odczytywanie odkompresowanych na nowo
danych komórkowych do ujęcia wskazanego celowo u R+
HLT // Kompresuje zatrzymanie cykli wymuszając pełen zapad systemu by zatwierdzić brak
wyjść powrotnych pod odczyt
```

9.3 Generowanie ciągu Fibonacciego

Program precyzyjnie wykonuje skokowe powiązania liczenia dla układu bazowego opierając testy o wymierny licznik podpięty środowiskowo dla indeksów na R+. Pod sam koniec bloku docelowego maszynowe zapory pozostawiają sprawnie przypięte układy przeliczonych cyfrowo do wektorów na wymiarach R-.

```
LLI R- 0 // Pusta baza alokacji wyjściowego obwodu na logikę o parametrze startowym
LLI R0 1 // Docelowe operacyjne powierzone testom u góry dla rzutu parametrów
jedynkowych
LLI R+ 8 // Osadzenie górnego marginesu śledzenia i nadzoru przez licznik przeskoku
iteracji
PSH R+ // Odłożenie tymczasowych ujęć wyjściowych o wskaźniki góry ze stosu na dół
cykli
[-265712] PSH R0 // [docelowe celowniki] Wymuszone zadeklarowane zrzucane R0 dla
zachowania precyzji testów obwodowych
ADD R0 R- // Pociąga operację nad zrzutkami poprzez test wektorowy do nowych wyników
ciągu
POP R- // Oddaje wyjętą wprost oryginalną z logiki wyjściową powrotną logikę do odrzutu
o R-
POP R+ // Wywleka odliczony wskaźnik skoków pod odtworzone miejsce na liczenie ram
PSH R0 // Izoluje obwody przeliczonego na nowo zestawienia poprzez uchronienie nowych
testów z R0 w dół
LLI R0 1 // Manipulacyjnie blokuje rejestr przez powołane 1 do wykonania arytmetycznej
degradacji testów we wskaźnikach R0
SUB R+ R0 // Testowe zmniejszenie ujęć kroków operacji poprzez odbicie wyjściowe
narzutowe u wierzchu
POP R0 // Wstrzykuje chronione pierwotne zachowania wskaźnika komórkowego i powraca
stery celom od R0
PSH R+ // Ponawia chronienie skokowe przy odliczaniu testów chroniąc ujęcia w celowości
u stosu
LLI R+ -265712 // Śle wektor dla odniesienia wejścia kierującego z R+ prosto w
początkowy cel testowy
JMP R+ // Opcjonalnie zawraca wyjściowo wyjęte ze standardowego kroku uwarunkowania
obwodu
POP R+ // Uprząta resztki alokacyjne u wyjścia zachowując wymierne stabilności
środowiska po końcach testów w pętli
HLT
```

10. ZABEZPIECZENIA I OBSŁUGA BŁĘDÓW

Monitor granic sprzętowych	Wymóg zdarzenia i uwarunkowania obwodu	Działanie operacyjne logiki powstrzymującej
Limit bezpieczeństwa Watchdog	Wymiar zaplecza stepsExecuted wykracza na limity po osiągnięciu poziomu 1500 obrotów środowiska	Zrzuca zabezpieczająco przebieg w bezpieczne tryby awaryjne wpisując flagę logiki z dopiskiem: [TIMEOUT]
Monitor przestrzeni wejścia Stack Guard	Adres ujętego alokatora SP pikuje poza bazy graniczne po dnie od strony limitu [-265,720]	Kompletnie zawiesza blokowanie procesora osadzając dla testów powiadomienie awaryjne ujęcia: [KERNEL PANIC]
Zabezpieczenia licznika Program Counter	Wektor wskazujący dla adresów rzuca lokalizacje poza celami z puli komórek i rozbija ramy	Zawiesza ścieżki sterowania awaryjnego poprzez oddanie logowania błędów: [KERNEL PANIC]
Fizyczne rozbicie komór dekodera obwodów	Obracany adres polecenia z indeksów po id uniemożliwia poprawne powiązania u kluczy tabel dekodera	Trwale blokuje komendy rzucając pełnymi kodami typu ścisłego błędu oznaczanego jako standardowy model błędu RuntimeException
Odrzucenia weryfikacji warstwy walidatora logiki	Cykle środowiska wychwytuja błędy parametryczne o braku synchronizacji rozmiarowej dla polecenia na funkcji z walidacją	Uchrania asembler oddając sygnały pozycjonujące poprzez `[ERROR Ln]` i puszcza system odrzucając awaryjne rzędy tekstowe bez błędów pod RAM

11. WYMAGANIA TECHNICZNE I OGRANICZENIA ŚRODOWISKOWE

Architektoniczny Moduł Komponentu Środowiskowego	Skategoryzowane minimalne zasoby wymagane z uwarunkowaniem weryfikacji docelowej specyfikacji wymiaru startowego operacji maszyny	Adnotacje i funkcjonalności oddelegowane względem celowego oparcia operacyjnego z bazy weryfikacji
Fundamenty środowiska z bibliotek Java Development Kit	Odblokowane dla systemu testów u ujęciach z wymiarom SE SDK 17 lub zaktualizowanych pułapach architektur nadrzędnych u wejścia bazowego maszyn i warstw obróbki kodu.	Stanowią wsparcie dla ujęcia modułowego silnika weryfikując rdzenne działania dla struktur z parametrami operacyjnymi bez obciążeń ze wsparcia starych bibliotek kompilacyjnych.
Platformy logiki wyświetlającej interfejs JavaFX	Oczekiwane powiązania wektorów o ujęcia z wersji OpenJFX w specyfikacji stabilnych z modeli rdzennych 21.0.2 jako wymóg bezwartunkowy dla poprawnych ujęć graficznych na pulpitych stacji roboczych u badacza.	Opierają pełen wyciąg danych graficznych wspierając precyzyjne odnośniki dla ujęć pakietowych baz i okienek kontrolnych środowisk testowych dla wstrzykiwania kodu w matrycach widoku.
Obsługiwane sprzęty fizyczne docelowych układów bazowych systemów rzutowania i sterowania pod architekturę hostów roboczych	Ukierunkowane wsparcie obejmuje standardowe wieloplatformowe osadzenie stacji Desktop wykorzystujących pule z Windows, środowisk testowych pod platformami Linux czy w pełni obsługiwanych przestrzeni dla natywnych układów systemów maszyn po warstwy obsługi systemów uniksowych z seriami Apple macOS.	Niweluje błędy i spaja poprawność obsługiwania całego zaplecza dzięki maszynom ujęcia krzyżowego na przestrzeniach maszyn optymalizowanych logicznie przy użyciu wirtualnego hosta z procesów wielowarstwowych operacji poprzez wyizolowane instancje powłok Java.

Dokumentacja wygenerowana automatycznie dla Ternary SoC Emulator. Data kompilacji:
02.06.2026 | Projekt docelowy: Ternary SoC Emulator | Przestrzeń pakietów: javaproject